

Lab 0 report

卢天姿 24300130073
新闻 + 人工智能

September 17, 2026

Contents

1	学习 Git 的基本用法	2
1.1	你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？	2
1.2	思考一下，Git 为什么要设计“暂存-提交”两个步骤？	2
1.3	git branch 和 git branch -a 的区别是什么？查阅资料并回答。	2
2	建立个人仓库 (STEP1)	2
3	为什么要学习 Git	3
4	Git 分支管理 (STEP2)	4
4.1	冲突是什么？	4
4.2	如何解决冲突	5

1 学习 Git 的基本用法

1.1 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

A：有过的，也是在 GitHub 上，但是是分工分块去做实验的不同部分，最后 merge 在一起。如果遇到需要承上启下的部分，往往需要等待上一个人做完提交到 Github 上再进行延续，这样大大降低了效率，无法做到同时协作。

1.2 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

A：当我内部有多个版本更新的时候可以自行控制提交哪些，哪些不被提交，方便做精细的版本控制。Git 将“暂存”和“提交”分成两个步骤，让我们在正式保存一个版本之前，还有挑选和检查的机会。在实际修改代码时，我们可能同时改了很多内容，但这些修改不一定都属于同一个任务。通过 `git add`，我们可以只把这次想提交的部分放进暂存区，暂时不提交其他还没完成或者不相关的修改。确认暂存区中的内容没有问题后，再使用 `git commit` 将它们保存成一个正式版本。这样可以使每次提交的内容更加清楚、完整，也方便以后查看修改记录、寻找问题或者退回到之前的版本。

1.3 `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

A：`git branch` 只列出本地分支，并使用星号 * 标记当前所在的分支；`git branch -a` 中的 `-a` 是 `--all` 的缩写，它会同时列出本地分支和远程跟踪分支。远程跟踪分支通常以 `remotes/origin/` 开头。

2 建立个人仓库（STEP1）

1. 从官方版本的 template 挪到自己的 github，create a new repository 后下载到本地；
2. 完成 `main.c` 当中的 TODO，将 `printf` 中的语句修改为 “I have completed the first lab of ICS!”，进行 `clean` 编译验证是否存在错误；

```
● (base) sophialu@MacBook-Air-4 ics-Lab0 % make clean
make
./main
make clean
rm -f main.o main
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
I have completed the first lab of ICS!
rm -f main.o main
```

Figure 1: 完成 TODO 句子修改

3. 验证正确后进行 `commit` 基本流程，`push` 到仓库中去；

```

• (base) sophialu@MacBook-Air-4 ics-Lab0 % git status
git add main.c
git commit -m "revise TODO and commit"
git push origin main
位于分支 main
您的分支与上游分支 'origin/main' 一致。

尚未暂存以备提交的变更:
  (使用 "git add <文件>..." 更新要提交的内容)
  (使用 "git restore <文件>..." 丢弃工作区的改动)
      修改:   main.c

修改尚未加入提交 (使用 "git add" 和/或 "git commit -a")
[main 002406f] revise TODO and commit
Committer: lu <sophialu@MacBook-Air-4.local>
您的姓名和邮件地址基于登录名和主机名进行了自动设置。请检查它们正确
与否。您可以对其进行设置以免再出现本提示信息:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

设置完毕后, 您可以用下面的命令来修正本次提交所使用的用户身份:

    git commit --amend --reset-author

1 file changed, 1 insertion(+), 2 deletions(-)
Enter passphrase for key '/Users/sophialu/.ssh/id_ed25519':
枚举对象中: 5, 完成.
对象计数中: 100% (5/5), 完成.
使用 8 个线程进行压缩
压缩对象中: 100% (3/3), 完成.
写入对象中: 100% (3/3), 353 字节 | 353.00 KiB/s, 完成.
总共 3 (差异 1), 复用 0 (差异 0), 包复用 0 (来自 0 个包)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:fdulumos-wq/ics-Lab0.git
   22eb43e..002406f  main -> main

```

Figure 2: 把修改 commit

3 为什么要学习 Git

1. 方便版本迭代, 清晰明了, 而且能留存不同版本。 在实际的软件开发中, 项目通常不是一次完成的, 而是会经历不断的修改、完善、测试和发布。Git 可以记录项目在不同时间点的状态, 每一次 commit 都相当于为项目保存了一个明确的版本节点。因此, 当我们不断增加功能、修改代码或修复 Bug 时, 可以清晰地看到项目是如何一步步演变的, 也可以在出现问题时回到之前的版本, 而不需要手动保存大量的“ver123”等文件。

更重要的是, Git 记录的不只是某一个最终结果, 而是完整的版本演进过程。结合 Gitflow 的思路, 一个项目还可以通过不同的分支来管理不同阶段的开发, 例如在 feature branch 中开发新功能, 在 develop 中整合开发内容, 在 release 阶段准备发布, 并通过 hotfix 处理已经发布版本中的紧急问题。这样, 项目的不同变化可以被组织在清晰的开发流程中, 而不是所有修改都混杂在同一个版本里。

2. 每次有 commit message, 方便快速浏览、查找信息。 Git 的另一个重要价值在于, 它不仅记录“代码发生了变化”, 还可以记录“这次变化是为了什么”。每次 commit 都需要配合 Commit Message, 用简洁的文字说明本次提交的目的。规范的 Commit Message 可以提供更多的历史信息, 帮助我们快速浏览项目的修改记录; 同时, 还可以根据关键词过滤 commit, 从而快速查找某一类修改。例如, 当我们只想查看某个版本新增了哪些功能, 或者想寻找过去与某个 Bug 相关的修改时, 就不需要逐个检查所有代码, 而可以直接通过提交记录进行定位。

因此, Commit Message 实际上是在给项目的变化添加“解释”。随着项目不断迭代, 这些提交记录会形成一条清晰的项目历史, 甚至可以进一步自动生成 Change Log, 用来总结不同版本之间新增了什么功能、修复了什么问题以及发生了哪些重要变化。

3. 方便协作，各司其职，能够实现不同任务同步推进且互不影响。当项目从个人开发变成多人协作之后，Git 的价值会更加明显。一个项目往往会同时存在多个任务，例如一个人开发新功能，一个人修复 Bug。如果所有人都直接修改同一份代码，就很容易互相覆盖修改，或者因为不同任务同时进行而产生混乱。

Git 通过 branch、merge 等机制，将不同任务的开发过程进行隔离。团队成员可以针对自己的任务建立独立的分支，在不影响主线代码的情况下完成开发；任务完成后，再将经过验证的修改合并回相应的分支。Gitflow 进一步将这种协作过程组织成不同的分支和开发阶段，使新功能开发、版本发布和紧急修复能够按照不同的流程进行。

因此，Git 不只是解决“如何保存代码”的问题，更是在解决“多人如何共同修改同一个项目”的问题。每个人可以负责自己的任务，不同任务可以并行推进，同时又可以通过统一的版本管理和合并机制，最终将各部分工作整合成一个完整的项目。

4 Git 分支管理 (STEP2)

4.1 冲突是什么？

1. 新建并切换到 feature 分支，修改 feature 内部 main.c 的 print 内容（"This comes from FEATURE branch!"），重新 commit 上去；

```
• (base) sophialu@MacBook-Air-4 ics-Lab0 % git switch -c feature
  切换到一个新分支 'feature'
• (base) sophialu@MacBook-Air-4 ics-Lab0 % git branch
  * feature
    main
• (base) sophialu@MacBook-Air-4 ics-Lab0 % make
./main
make clean
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
This comes from the FEATURE branch!
rm -f main.o main
• (base) sophialu@MacBook-Air-4 ics-Lab0 % git add main.c
  git commit -m "update on creating a feature branch"
  git push -u origin feature
[feature 0a72cae] update on creating a feature branch
```

Figure 3: feature 分支

2. 切换回 main 主分支，修改 main 内部 main.c 的 print 内容（"This comes from main branch."），再次 commit；

```
• (base) sophialu@MacBook-Air-4 ics-Lab0 % git switch main
  切换到分支 'main'
  您的分支与上游分支 'origin/main' 一致。
• (base) sophialu@MacBook-Air-4 ics-Lab0 % make
./main
make clean
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
This comes from the main branch.
rm -f main.o main
• (base) sophialu@MacBook-Air-4 ics-Lab0 % git add main.c
  git commit -m "update on changes on main branch to make a conflict"
[main 00efae6] update on changes on main branch to make a conflict
  Committer: lu <sophialu@MacBook-Air-4.local>
```

Figure 4: main 分支

3. 将两条时间线 merge 在一起，出现了合并冲突；

```

⊙ (base) sophialu@MacBook-Air-4 ics-Lab0 % git merge feature
自动合并 main.c
冲突（内容）：合并冲突于 main.c
自动合并失败，修正冲突然后提交修正的结果。

```

Figure 5: 终端 merge 报错

```

1  #include <stdio.h>
2
3  int main()
4  {
5      采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6      <<<<<< HEAD (当前更改)
7      printf("This comes from the main branch.\n");
8      =====
9      printf("This comes from the FEATURE branch!\n");
10     >>>>>> feature (传入的更改)

```

Figure 6: 程序报错

4.2 如何解决冲突

出现合并冲突本质上是因为两个分支对于代码的同一位置进行了不同修改，因此无法判断哪一个修改才是要保留的。就会出现Figure 6中的当前更改和传入的更改。为了将feature分支合并到main分支，我阅读了git merge部分，也看到了Figure 6中 HEAD 上方提示的修改模式，最终选择**保留双方更改**这种修改方式（见Figure 7），修改后执行git add main.c 和 git commit 完成合并提交，运行成功了！

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("This comes from the main branch.\n");
6      printf("This comes from the FEATURE branch!\n");
7  }

```

Figure 7: 保留双方更改，把两个 printf 的内容都加入 main 分支，feature 分支仍旧不变。

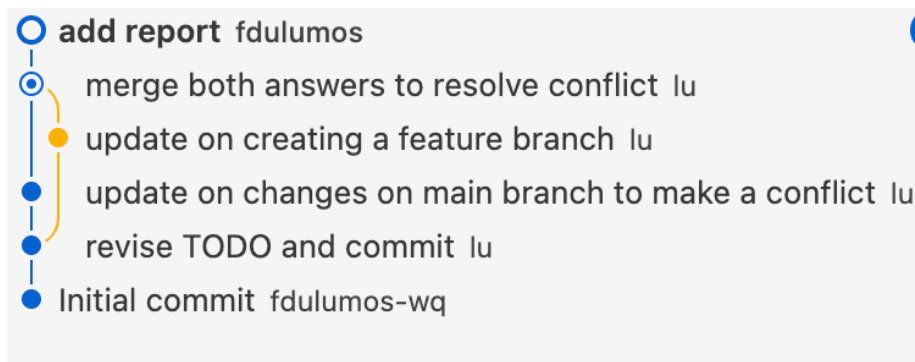


Figure 8: 提交并成功运行